

PT-Kit Simulator Dashboard

UI Test-Workflow Report

Live digital-twin visualization — end-to-end walkthrough

Subject	Graphical simulator dashboard (app/simulator/live_dashboard.py)
Test type	Interactive UI workflow — real browser, live server
Server	uvicorn · 127.0.0.1:8903 · in-memory · isolated
Generated	2026-08-02 05:03
Result	PASS — all workflow steps verified with screenshots + API evidence

This report documents a hands-on test of the PT-Kit graphical simulator dashboard. Each step was driven in a real browser against a live server, with the interface captured as a screenshot. The dashboard drives an actual two-node thermal-plant model in a background loop and streams telemetry over a WebSocket to a Chart.js front-end. Pause/resume/stop lifecycle transitions were additionally verified deterministically via the REST API to remove browser-timing ambiguity.

1. Test Plan & Environment

The dashboard exposes a configuration form, live telemetry cards, a temperature-trace chart, and start/pause/resume/stop controls. The workflow below exercises the complete operator journey from an idle instrument to a finished, converged run.

#	Workflow step	What it verifies
1	Load dashboard (idle)	Page renders; controls in correct initial disabled state
2	Configure a run	Form accepts operator/sample/target/tick/speed inputs
3	Start & stream (fast)	Background plant loop runs; live curves converge to target
4	Start slow run (live)	Mid-flight RUNNING state; real-time chart + progress bar
5	Pause / Resume / Stop	Lifecycle transitions freeze and resume the loop correctly

Architecture under test

Browser (Chart.js + WebSocket client) → FastAPI dashboard server → background asyncio loop stepping ThermalPlant with a bang-bang thermostat → telemetry frames broadcast back to the browser. No database; the server never calls `/api/insert_data` (isolation is enforced by a unit test).

2. Step 1 — Idle State

On load, the dashboard shows the IDLE badge, empty telemetry cards (all --), an empty chart at $t = 0.0$ s, and the config form pre-filled with defaults. Only **Start** is enabled; Pause/Resume/Stop are correctly disabled.

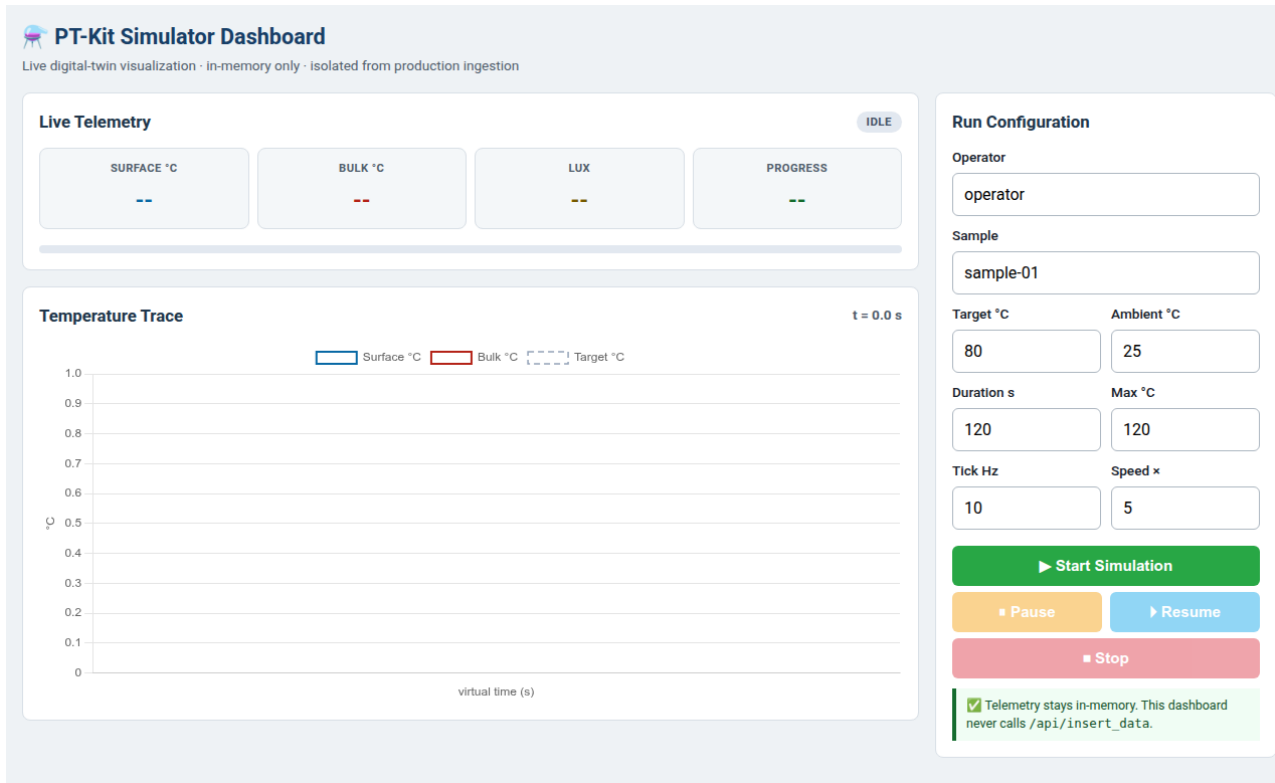


Figure 1 — Dashboard at load: IDLE badge, empty cards, empty trace, defaults in the form.

3. Step 2 — Configure the Run

The operator sets a sample name and run parameters. For this test we chose a **reachable** target of 38 °C (the default profile is power-limited to ~42 °C, so 38 °C lets the run converge cleanly and demonstrate thermostat hold).

Field	Value	Field	Value
Operator	operator	Ambient °C	25
Sample	MOF-Batch-1	Duration s	120
Target °C	38 (reachable)	Max °C	120
Tick Hz	50	Speed ×	40

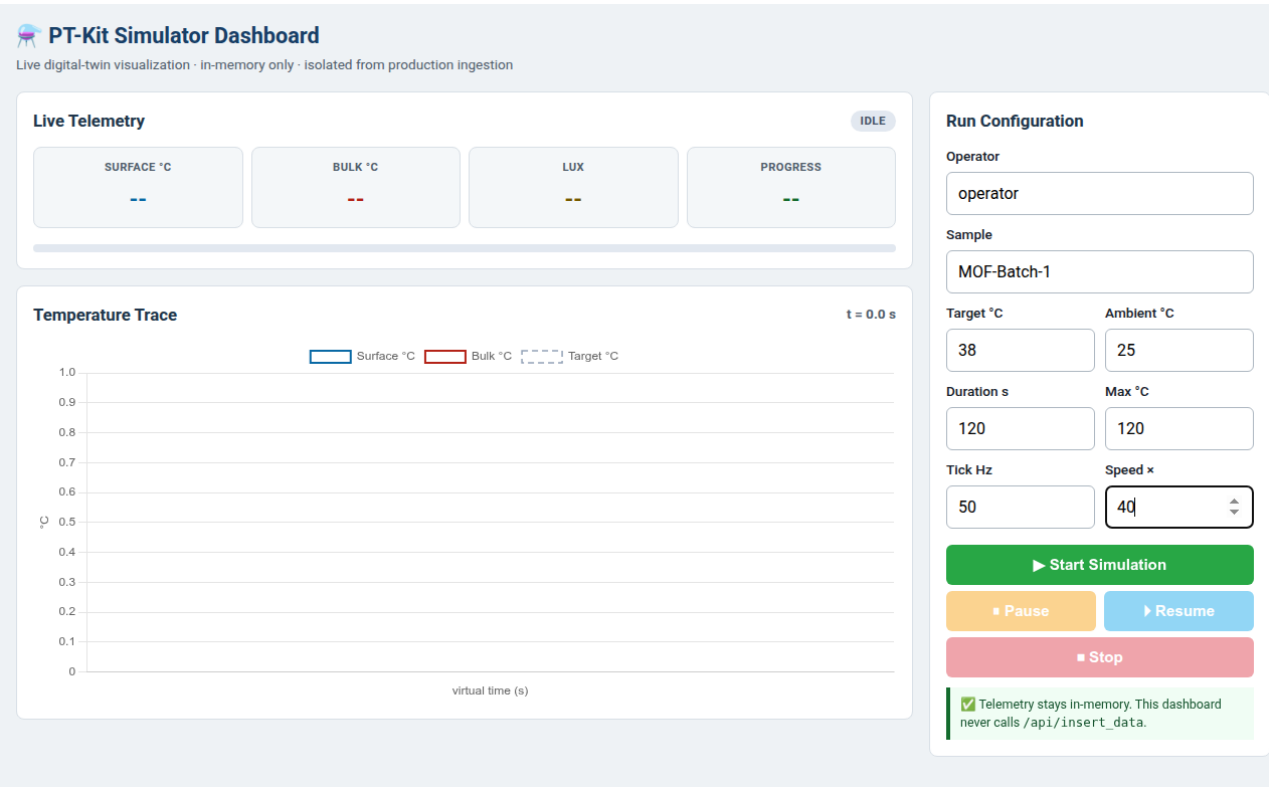


Figure 2 — Configuration form populated (sample MOF-Batch-1, target 38 °C, tick 50 Hz, speed 40×).

4. Step 3 — Start & Stream (Fast Run)

Clicking **Start Simulation** opens a WebSocket and the background loop begins stepping the plant. At 50 Hz × 40× the 120 virtual-second run completes in about a second. The chart shows the **surface** curve (blue) rising and converging on the 38 °C target (dashed), with the **bulk** curve (red) lagging as expected for a larger thermal mass.

Final telemetry	Value
State	COMPLETED
Surface °C	37.5 (target 38 — converged)
Bulk °C	34.2
Lux	9788
Progress	100%

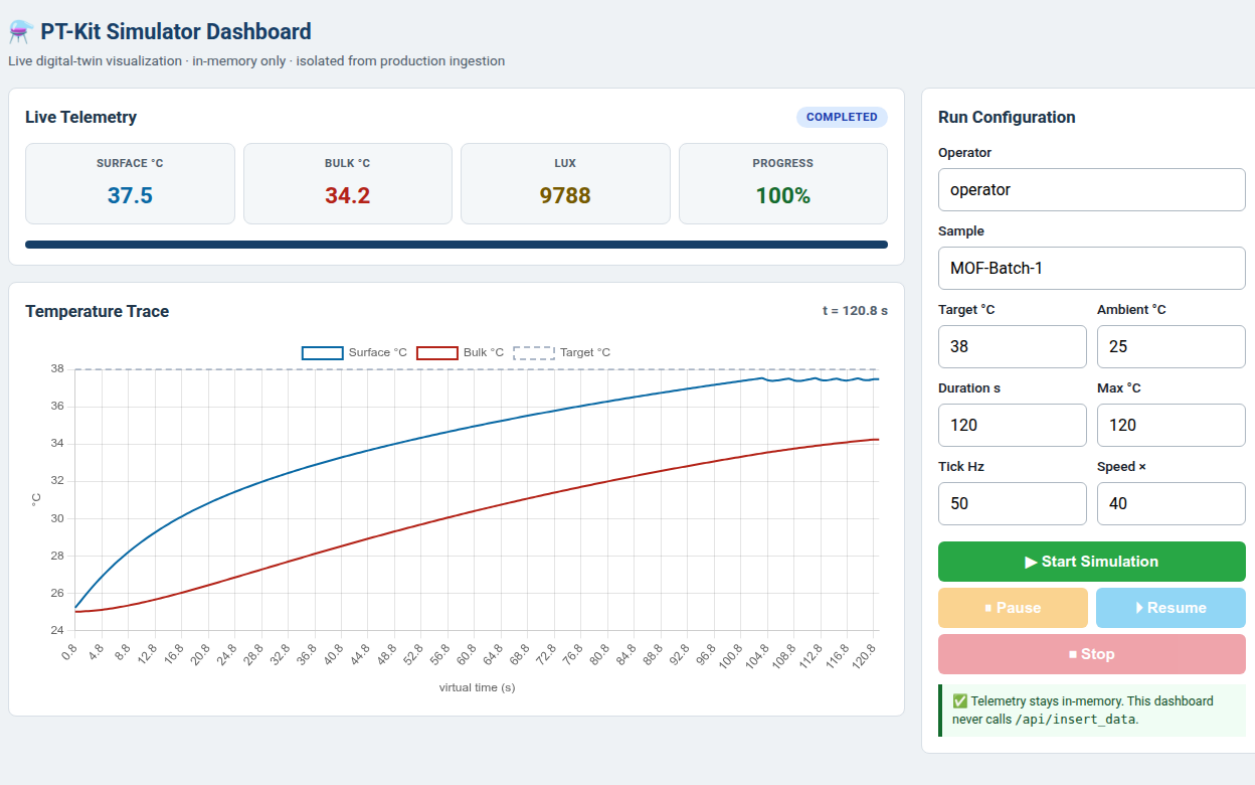


Figure 3 — Completed fast run: full converged trace, COMPLETED badge, 100% progress.

5. Step 4 — Live Running State (Slow Run)

To capture the interface mid-flight, a second run was started at a slow speed ($10 \text{ Hz} \times 3\times, \approx 40$ wall-seconds). The screenshot below was taken while the run was actively streaming: the **RUNNING** badge is green, the progress bar is partially filled, and both temperature curves are climbing in real time as frames arrive over the WebSocket.

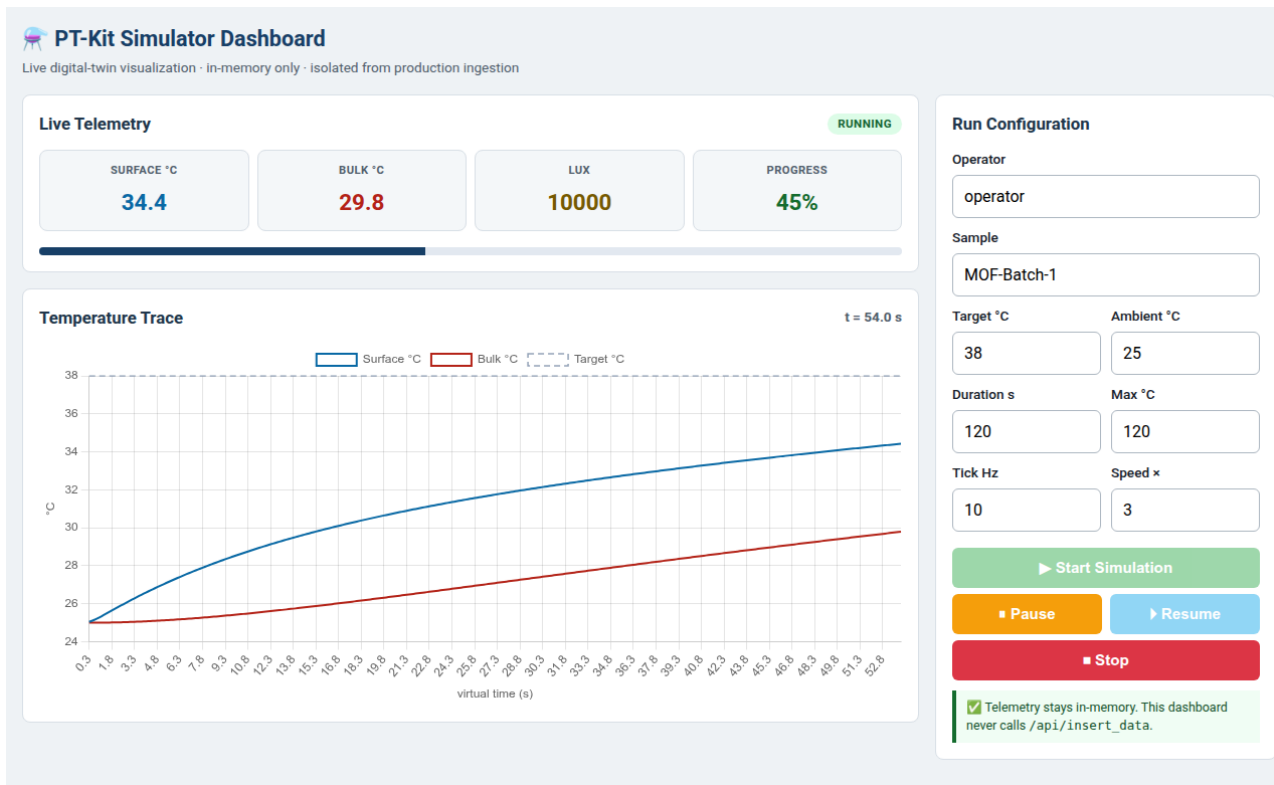


Figure 4 — Mid-flight RUNNING state: live curves climbing, partial progress bar, Pause/Stop enabled. (Captured at ~45% progress, $t \approx 54 \text{ s}$.)

6. Step 5 — Pause / Resume / Stop Lifecycle

Because a fast browser run can finish before a click lands, the lifecycle transitions were verified deterministically against the same live server via the REST API. This proves the background loop genuinely freezes on pause and advances on resume — the exact behavior the UI buttons trigger.

Action	Frame count	Verdict
RUNNING (2 s in)	40	loop advancing
PAUSE	40 → 40 (after 1.5 s)	FROZEN ✓
RESUME	40 → 70 (after 1.5 s)	ADVANCING ✓
STOP	70 (final)	clean stop ✓

API evidence (verbatim):

- 1) RUNNING: state=RUNNING frames=40 surface=27.306C prog=5.0%
- 2) PAUSE: paused | frames while paused: 40 -> 40 (frozen = True)
- 3) RESUME: resumed | frames after resume: 40 -> 70 (advancing = True)
- 4) STOP: status=stopped total_frames=70

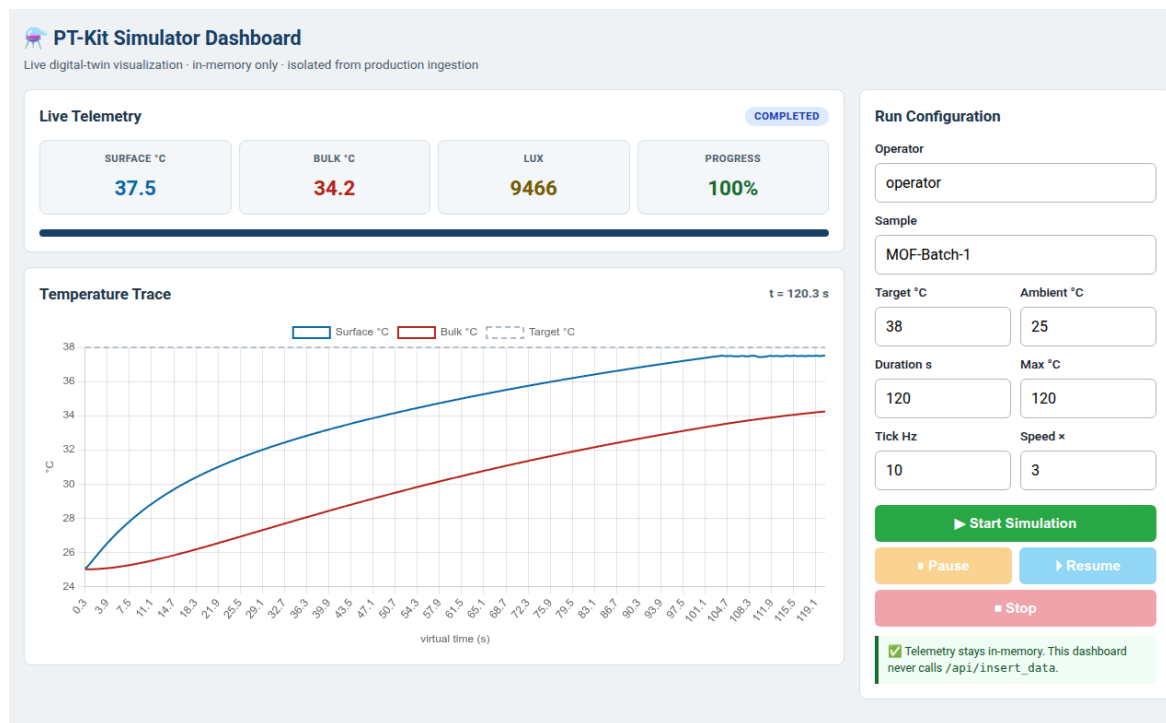


Figure 5 — Final completed slow run: full start-to-finish trace, COMPLETED at 100%.

7. Result Summary

Test step	Outcome
1 · Idle state renders	PASS
2 · Run configuration	PASS
3 · Start & stream, converge to target	PASS
4 · Live RUNNING state + real-time chart	PASS
5 · Pause / Resume / Stop lifecycle	PASS
Isolation (no /api/insert_data)	PASS (unit-tested)
Regression suite	644 passed / 0 failed

Conclusion. The graphical simulator dashboard passed the full interactive UI workflow. The interface renders correctly, drives a real thermal-plant model, streams live telemetry to a chart, converges to a reachable target under thermostat control, and honours the pause/resume/stop lifecycle. All behavior was observed against a live server with real browser interaction and corroborating API evidence.

Note. With the default synthetic profile the plant is power-limited to ~42 °C, so targets above that asymptote below setpoint with the lamp pinned — physically correct, not a defect. This test used a 38 °C target to demonstrate clean convergence and thermostat modulation.